

WEEK 7 – WEB APPLICATION VAPT ASSESSMENT REPORT

Vulnerability Assessment and Penetration Testing with Burp Suite — DVWA Lab

Student: PAILACHUKKA SUSMITHA

DG Interns Hub – Cybersecurity Internship

Target: Ubuntu Linux (DVWA) — 192.168.56.101:4280

Attacker/Testing Machine: Kali Linux

Primary Tool: Burp Suite Community Edition | Supporting Tool: Gobuster

Table of Contents

- 1. Executive Summary
- 2. Introduction
- 3. Objectives
- 4. Lab Environment
- 5. Tools Used
- 6. Methodology
- 7. Task 22 – Burp Suite Setup & Interception
- 8. Task 23 – Reconnaissance & Web Application Mapping
- 9. Task 24 – Vulnerability Testing
- 10. Task 25 – Directory & Endpoint Discovery
- 11. Burp Intruder Configuration & Limitation
- 12. Vulnerability Summary
- 13. Remediation & Security Recommendations
- 14. Conclusion
- 15. Evidence / Screenshot References

1. Executive Summary

This report documents Week 7 of the DG Interns Hub Cybersecurity Internship, a practical Vulnerability Assessment and Penetration Testing (VAPT) exercise focused on web application security. Testing was performed using Burp Suite Community Edition, complemented by the Gobuster directory enumeration tool, against a locally hosted instance of the Damn Vulnerable Web Application (DVWA) running on an Ubuntu virtual machine.

The assessment covered proxy-based interception and traffic analysis, application reconnaissance and site mapping, manual vulnerability testing for SQL Injection and Reflected Cross-Site Scripting, authentication testing against default credentials, parameter and cookie tampering, OS command injection testing, and directory/endpoint discovery. Six categories of security weaknesses were identified and are documented with evidence, risk ratings, and remediation guidance in this report.

All testing described in this report was performed exclusively against an authorized local DVWA lab environment owned and operated by the student. No public or third-party systems were tested at any point.

2. Introduction

Vulnerability Assessment and Penetration Testing (VAPT) combines systematic vulnerability identification with controlled exploitation techniques to evaluate the security posture of an application. This week's assessment focused specifically on web application VAPT using Burp Suite, an industry-standard HTTP proxy and testing platform, against DVWA — an application intentionally built with common vulnerabilities for security training purposes.

3. Objectives

- Configure and use Burp Suite Community Edition as an HTTP interception proxy.
- Perform reconnaissance and map the DVWA application's structure and endpoints.
- Manually test for SQL Injection and Reflected Cross-Site Scripting (XSS).
- Perform authentication testing, including default credential testing.
- Test parameter and cookie tampering against client-controlled values.
- Test OS Command Injection in a controlled manner.
- Perform directory and endpoint discovery using Gobuster.
- Configure Burp Intruder for a login-testing demonstration.
- Document findings, risk levels, and remediation recommendations.

4. Lab Environment

Component	Configuration
Attacker / Testing Machine	Kali Linux
Target Machine	Ubuntu Linux
Vulnerable Application	DVWA (Damn Vulnerable Web Application)
Target IP	192.168.56.101
DVWA Port	4280
DVWA URL	http://192.168.56.101:4280
Burp Proxy Listener	127.0.0.1:8080
Virtualization	Windows host + Oracle VirtualBox + Kali Linux + Ubuntu Linux

The assessment was performed entirely within this isolated local lab environment.

5. Tools Used

Tool	Purpose
Burp Suite Community Edition	HTTP proxy, request interception, Repeater-based manual testing, site mapping, Intruder configuration
DVWA	Intentionally vulnerable web application used as the test target
Gobuster	Directory and endpoint discovery / enumeration
Kali Linux	Security testing platform
Ubuntu Linux	Target host running DVWA

Tool	Purpose
Oracle VirtualBox	Isolated virtual lab environment

PAILACHUKKA SUSMITHA

6. Methodology

The assessment followed a structured VAPT workflow, moving from environment setup through to documentation:

- 1. Lab setup — isolated Kali/Ubuntu VirtualBox environment.
- 2. Burp Suite configuration — proxy listener verification at 127.0.0.1:8080.
- 3. Application reconnaissance — browsing DVWA through Burp's built-in browser.
- 4. Request interception — capturing live HTTP traffic via Burp Proxy.
- 5. Site mapping — building a structural map of DVWA using Burp's Target/Site Map.
- 6. Manual vulnerability testing — SQL Injection and Reflected XSS.
- 7. Repeater-based testing — resending and modifying captured requests.
- 8. Authentication testing — invalid and default-credential login attempts.
- 9. Parameter tampering — modifying client-controlled request parameters.
- 10. Command injection testing — testing the DVWA Command Injection module.
- 11. Endpoint discovery — directory/file enumeration using Gobuster.
- 12. Evidence collection — capturing screenshots of each test and result.
- 13. Vulnerability documentation — recording findings with risk ratings.
- 14. Remediation recommendations — proposing practical fixes for each finding.

7. Task 22 – Burp Suite Setup & Interception

Burp Suite Community Edition was opened and configured as the primary interception proxy for this assessment. The Burp Proxy listener was verified at 127.0.0.1:8080, and Burp's built-in browser was used to access the local DVWA application so that all traffic would route through the proxy for inspection.

- Captured HTTP requests using Burp Proxy.
- Examined HTTP request and response data.
- Used Burp's HTTP history to review captured traffic.
- Sent requests to Burp Repeater for manual resending and analysis.
- Captured and analyzed a DVWA login request.

Login Testing

Invalid password tested: Username: admin, Password: wrongpassword — the invalid login attempt produced a failed-login response

Valid credentials tested: Username: admin, Password: password (DVWA default credentials)

The successful authentication response included:

```
HTTP/1.1 302 Found
Location: index.php
Set-Cookie: PHPSESSID=...
```

This demonstrated that the default DVWA credentials were accepted in the local lab environment.

Week 7 – Web Application VAPT | Page 6

8. Task 23 – Reconnaissance & Web Application Mapping

Using Burp Proxy, Burp HTTP History, and Burp's Target / Site Map, the DVWA application was browsed and systematically mapped to identify its available areas, functionality, and input surface.

- Browsed the DVWA application through Burp's built-in browser.
- Mapped available application areas and endpoints (e.g. login.php, security.php, instructions.php, vulnerabilities/modules).
- Identified login functionality and input fields.
- Identified GET/POST parameters used across the application.
- Examined application requests and responses for structural details.

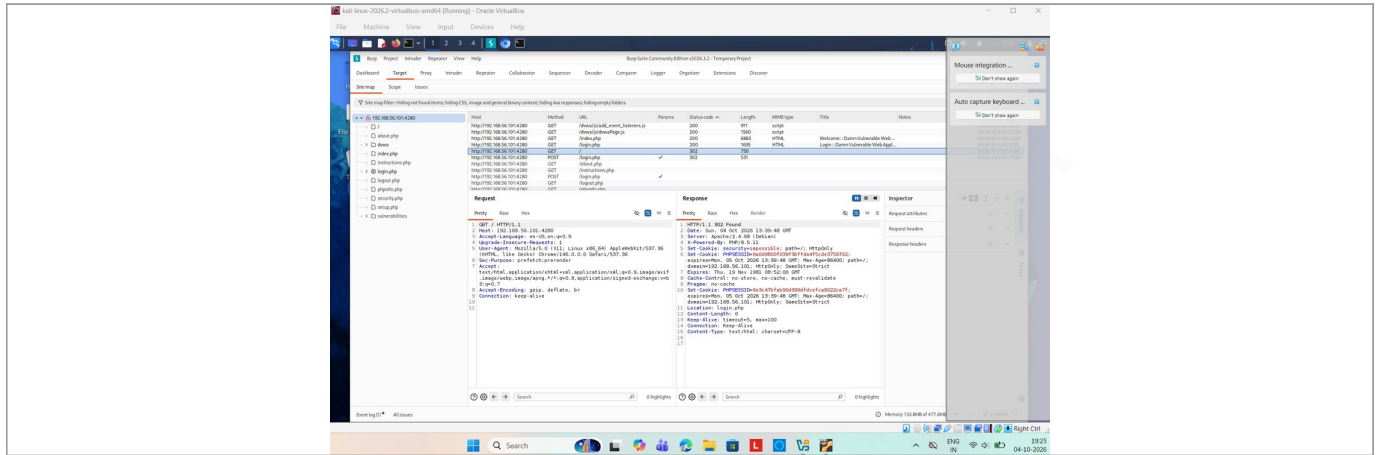


Figure 2: Burp Suite Site Map showing the mapped structure of the DVWA application, including login.php, logout.php, phpinfo.php, security.php, setup.php, and the vulnerabilities directory.

9. Task 24 – Vulnerability Testing

The following vulnerabilities and security issues were tested in the local DVWA environment, with DVWA's security level set to Low for controlled demonstration of each vulnerability class.

A. SQL Injection

SQL Injection is a vulnerability that occurs when untrusted input is inserted into a SQL query without proper validation or parameterization, allowing an attacker to alter the query's intended logic. The DVWA SQL Injection module was tested at Low security level.

Payload used: 1' OR '1'='1 #

At a high level, this payload closes the original quoted string early and appends a condition ('1'='1') that always evaluates to true, followed by a comment marker (#) that removes the remainder of the original query. This is intended to cause the query to return more data than the application's original, intended lookup.

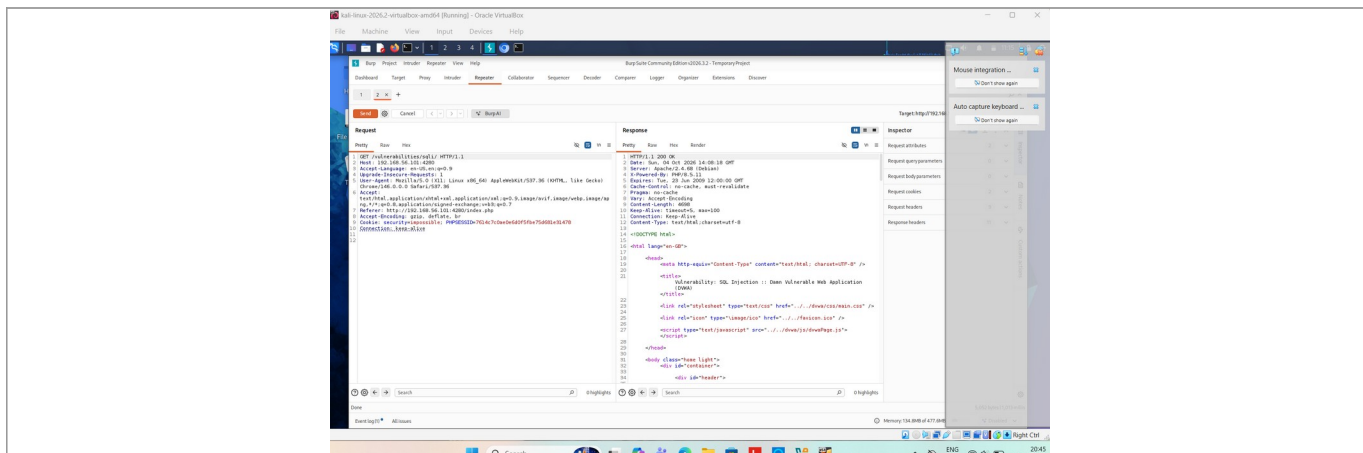


Figure 3: SQL Injection page request and response captured via Burp Repeater, confirming the request/response flow used for payload-based testing.

Security impact: SQL Injection can allow unauthorized access to data outside the application's intended scope, bypass of authentication logic, and in more severe cases, modification or deletion of database records, depending on the privileges of the database account in use.

Why parameterized queries matter: Parameterized queries ensure that user input is always treated as data and never as part of the executable SQL command, which eliminates this class of vulnerability regardless of what characters the input contains.

Remediation: Use parameterized queries / prepared statements, apply input validation, and use least-privilege database accounts.

B. Reflected Cross-Site Scripting (XSS)

Reflected XSS occurs when user-supplied input is included in the application's response without proper validation or encoding, causing injected script to execute in the victim's browser. DVWA's security level was set to Low for this test.

Payload used: `<script>alert(1)</script>`

The payload was submitted through the DVWA Reflected XSS page, and the application reflected it back into the page without sanitizing it. The script executed and displayed a browser alert popup, confirming the injected code ran as JavaScript rather than being rendered as plain text.

Popup displayed: 192.168.56.101:4280 says — 1

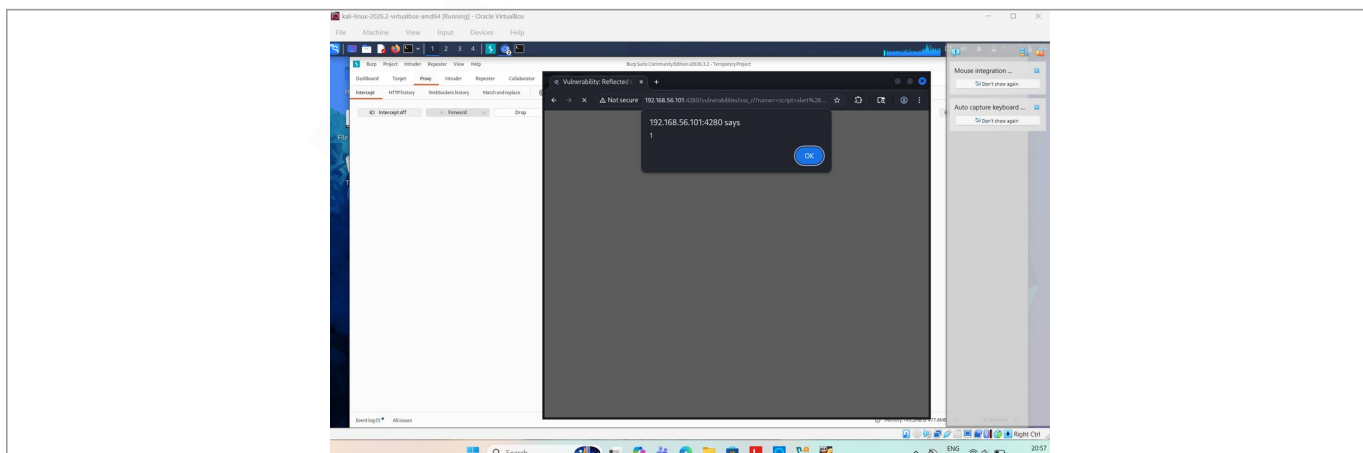


Figure 4: Reflected XSS payload successfully executed in DVWA, confirmed by the JavaScript alert popup displaying "1".

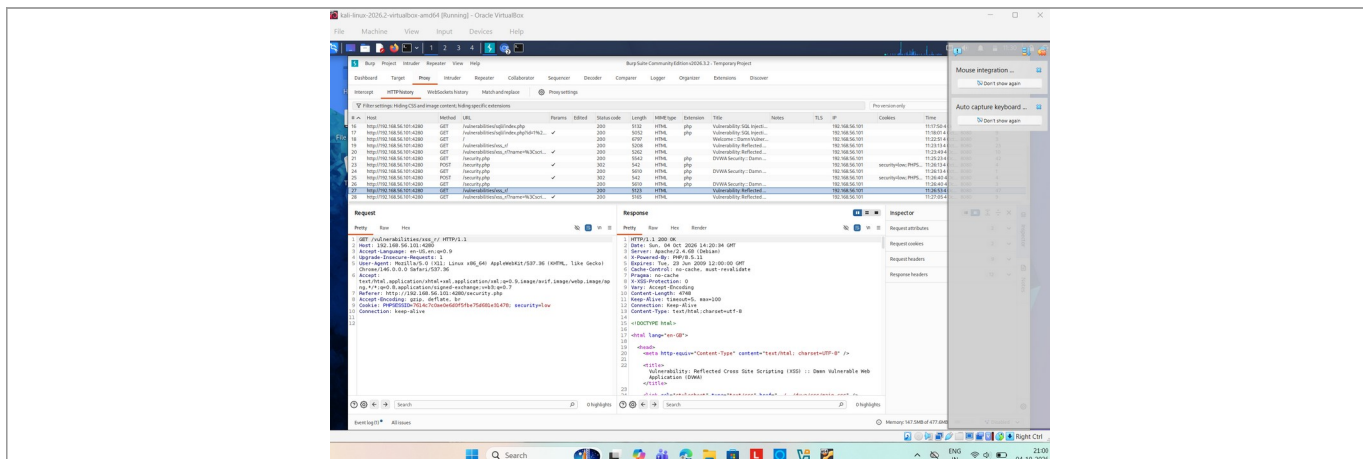


Figure 5: Burp Suite HTTP history showing the GET request to /vulnerabilities/xss_r/ and the 200 OK response confirming the reflected payload was processed by the application.

Security impact: Reflected XSS can be used to execute arbitrary JavaScript in a victim's browser, typically by tricking the victim into following a crafted link, potentially leading to session hijacking, credential theft, or actions performed in the victim's session context.

Remediation: Input validation, output encoding, Content Security Policy, and context-aware escaping of all user-supplied data before it is rendered in a response.

C. Authentication Testing / Weak Default Credentials

Authentication testing began with an invalid credential attempt, followed by a test using DVWA's known default credentials, to compare the application's behaviour between a rejected and an accepted login.

Invalid credentials tested: Username: admin, Password: wrongpassword — the application rejected the login

Default credentials tested: Username: admin, Password: password

The application accepted the default credentials and returned:

HTTP/1.1 302 Found
Location: index.php

along with a PHP session cookie (Set-Cookie: PHPSESSID=...), demonstrating that the default credentials were accepted in the DVWA lab environment.

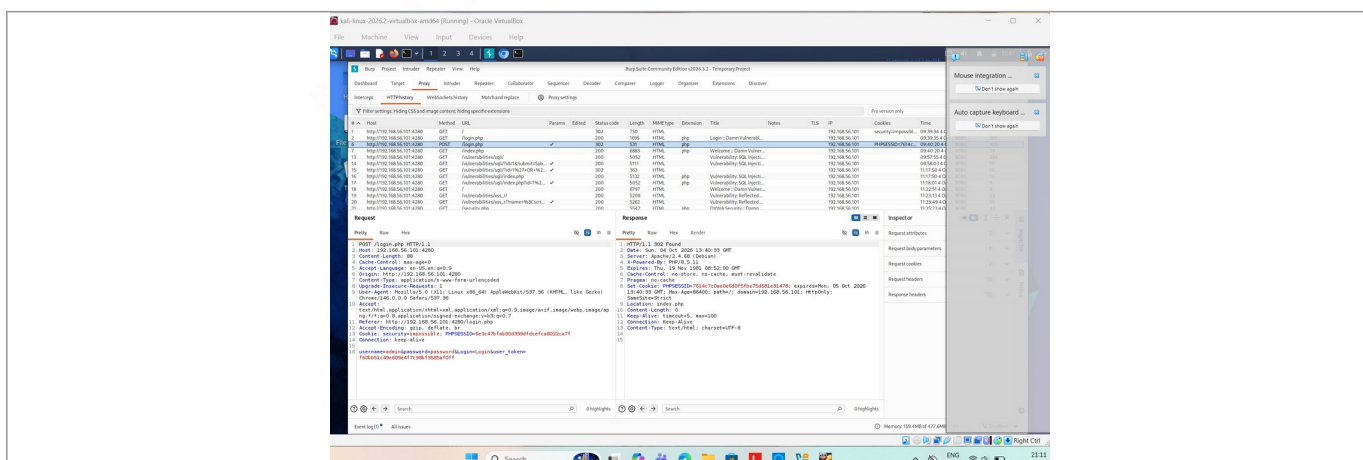


Figure 6: DVWA login request and response captured in Burp Proxy during authentication testing.

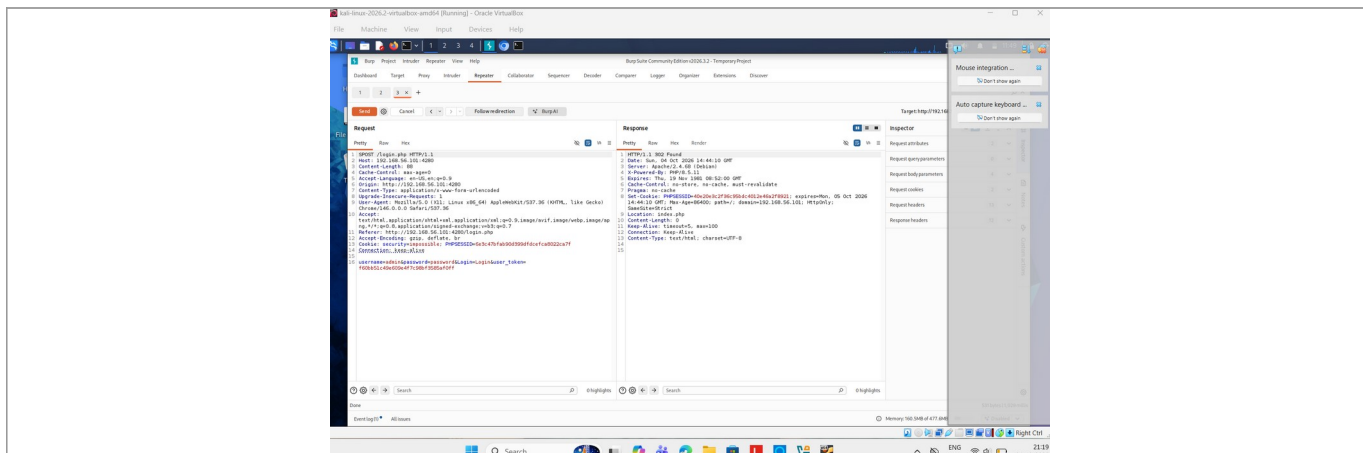


Figure 7: Default credentials (admin / password) resent via Burp Repeater, confirming repeatable acceptance via a 302 Found redirect to index.php.

Security risk: Default or weak credentials left unchanged in a production system allow trivial unauthorized access, since these credentials are widely known or easily guessed.

Remediation: Change default credentials, enforce strong password policies, implement account logout / rate limiting, use multi-factor authentication where appropriate, and apply secure session management.

D. Parameter / Cookie Tampering

A DVWA Command Injection request (POST /vulnerabilities/exec/) was captured and its security cookie was modified to observe whether the application's behaviour was controlled by a client-supplied value.

Original cookie: security=impossible;

Modified cookie: security=low;

After forwarding the modified request, the response changed accordingly and returned HTTP/1.1 200 OK, with the application's reported Security Level reflecting the modified cookie value rather than the server enforcing it independently.

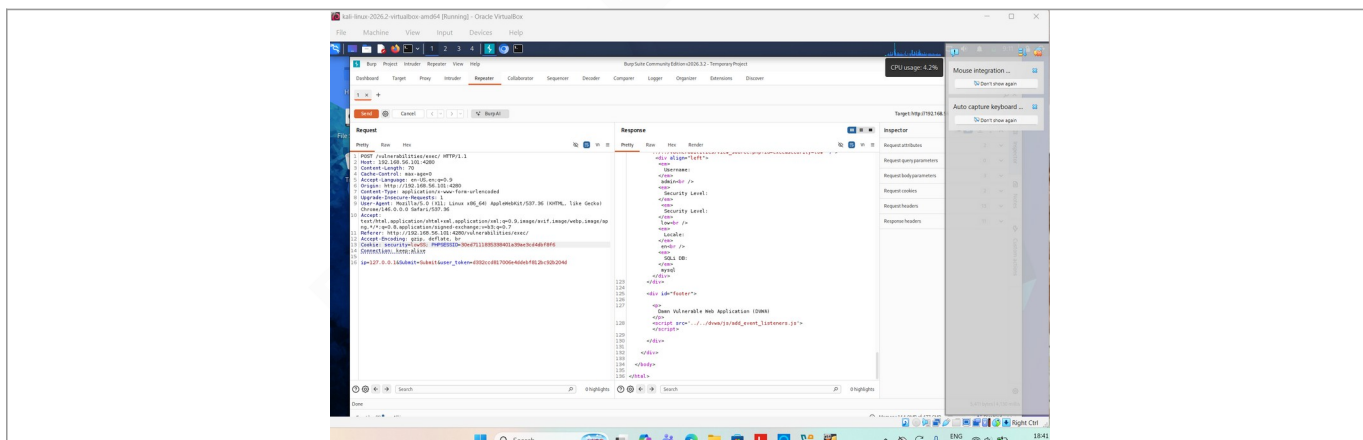


Figure 8: Burp Repeater response showing the application's reported Security Level reflecting the client-modified security cookie value.

A separate parameter modification was also tested on the same module:

Original parameter: ip=127.0.0.1

Modified parameter: ip=192.168.56.101

The modified request also returned HTTP/1.1 200 OK.

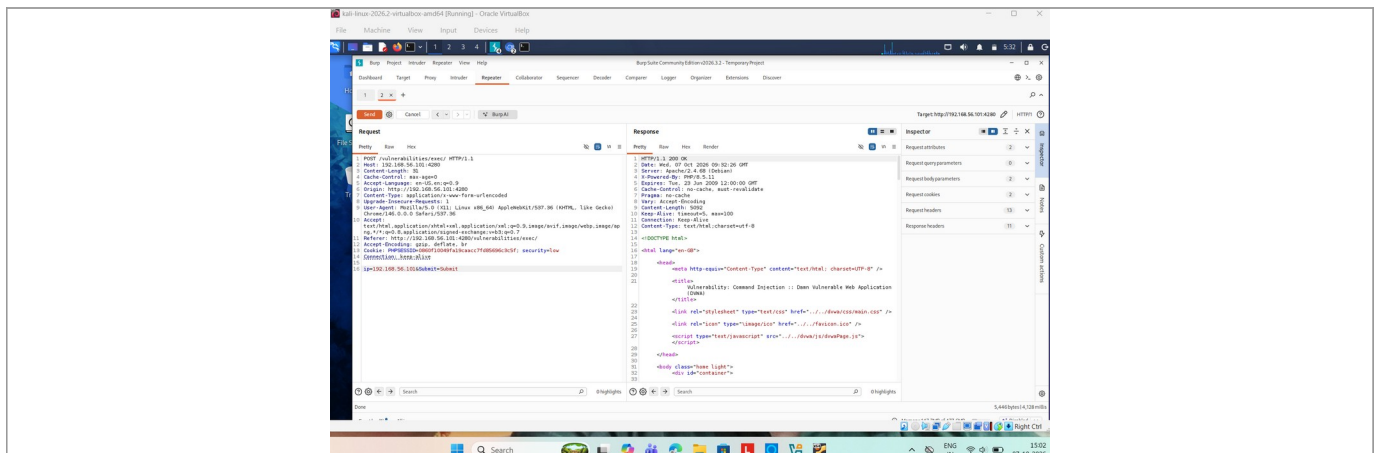


Figure 9: Burp Repeater request showing the modified ip parameter (192.168.56.101) and the resulting 200 OK response.

Why this matters: Client-controlled security decisions are unsafe because any value sent from the browser — including cookies — can be freely modified by the user before it reaches the server. Security-sensitive logic (such as which protection level is active) must never rely on trusting client-supplied values without server-side validation and authorization checks.

Remediation: Enforce security-relevant settings and authorization decisions entirely server-side, validate all incoming parameters regardless of source, and never treat client-controlled cookies or parameters as trusted security indicators.

E. OS Command Injection

The DVWA Command Injection module was tested. A normal request (ip=127.0.0.1) was first observed, followed by a command-injection payload sent through Burp Repeater.

Normal request: ip=127.0.0.1

Injection payload tested: 127.0.0.1; whoami

The crafted request was submitted via Burp Repeater and the application returned an HTTP 200 OK response. The full command output was not clearly visible within the captured response evidence available for this report, so this assessment does not claim to have confirmed the specific output of the whoami command — only that the payload was accepted and processed by the application without being rejected or sanitized at the input stage.

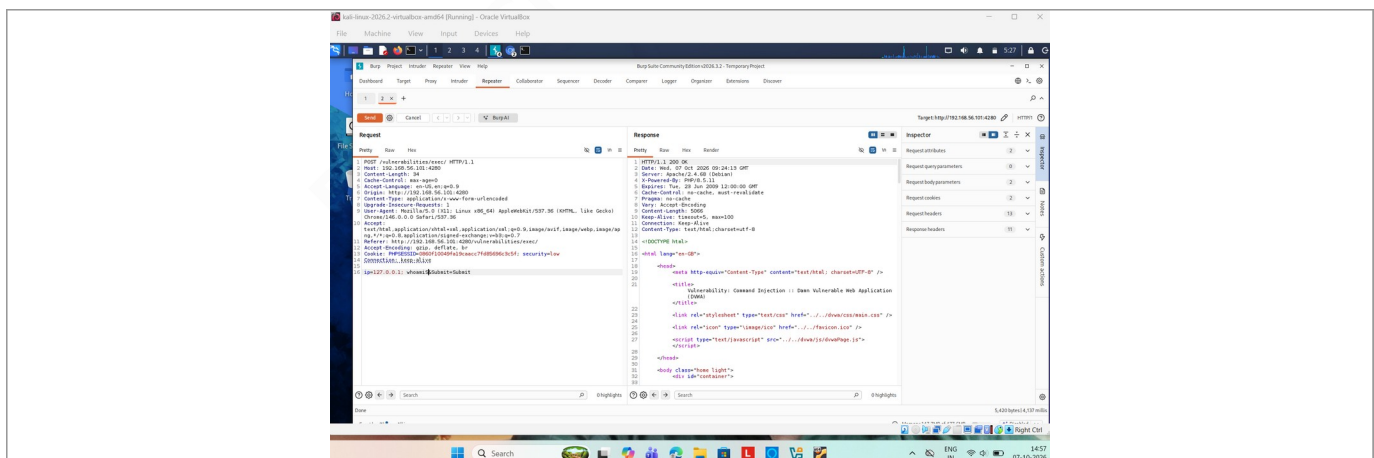


Figure 10: Command Injection payload (127.0.0.1; whoami) submitted via Burp Repeater to /vulnerabilities/exec/, returning an HTTP 200 OK response.

What OS Command Injection is: A vulnerability where unsanitized user input is passed to an operating-system command, allowing an attacker to append or inject additional commands that the underlying system will execute.

Potential impact: Unauthorized command execution, information disclosure, and in severe cases, full system compromise, depending on the privileges of the process executing the command.

Remediation: Apply strict allowlist input validation, avoid invoking shell commands with user input where possible, use safe language-level APIs instead of shell execution, and run processes with least privilege.

PAILACHUKKA SUSMITHA

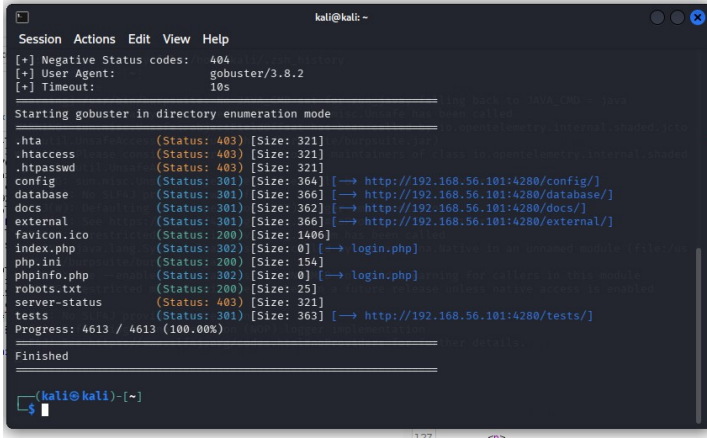
10. Task 25 – Directory & Endpoint Discovery

Gobuster was used to enumerate directories and files on the DVWA target that may not be linked from the application's visible navigation.

```
gobuster dir -u http://192.168.56.101:4280 -w /usr/share/wordlists/dirb/common.txt
```

Gobuster successfully discovered several endpoints and directories, including:

- /config/
- /database/
- /external/
- /login.php
- /phpinfo.php
- /server-status/
- /tests/



```
kali@kali: ~
Session Actions Edit View Help
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 321]
.htaccess (Status: 403) [Size: 321]
.htpasswd (Status: 403) [Size: 321]
config (Status: 301) [Size: 366] [→ http://192.168.56.101:4280/config/]
database (Status: 301) [Size: 366] [→ http://192.168.56.101:4280/database/]
docs (Status: 301) [Size: 362] [→ http://192.168.56.101:4280/docs/]
external (Status: 301) [Size: 366] [→ http://192.168.56.101:4280/external/]
favicon.ico (Status: 200) [Size: 1406]
index.php (Status: 302) [Size: 0] [→ login.php]
php.ini (Status: 200) [Size: 354]
phpinfo.php (Status: 302) [Size: 0] [→ login.php]
robots.txt (Status: 200) [Size: 25]
server-status (Status: 403) [Size: 321]
tests (Status: 301) [Size: 363] [→ http://192.168.56.101:4280/tests/]
Progress: 4613 / 4613 (100.00%)

Finished

(kali@kali)-[~]
$
```

Figure 11: Gobuster scan output showing discovered directories and endpoints on the DVWA target.

Purpose: Directory and endpoint discovery identifies application content that is reachable but not necessarily linked from the visible site navigation, helping to build a complete picture of the application's attack surface.

Why exposed endpoints matter: Endpoints such as phpinfo.php can disclose detailed server configuration information, and directories such as config/ or database/ may expose sensitive application internals if not properly access-controlled or removed in production.

Remediation: Remove unnecessary files and directories before deployment, apply access controls to any administrative or diagnostic endpoints that must remain, and follow production hardening guidance to minimize exposed surface area.

11. Burp Intruder Configuration & Limitation

Burp Intruder was configured for a login password-testing demonstration, to illustrate how automated payload-based testing would be set up against the DVWA login form.

Payload position: password=\$Password\$

Payloads prepared for the demonstration:

- wrongpassword
- Password
- password

Burp Suite Community Edition displayed a limitation/warning indicating that automated Intruder attack functionality was not

available in this edition. Therefore, Intruder configuration and payload preparation were demonstrated, but an automated attack was not executed. This report does not claim that a successful automated Intruder attack was completed.

12. Vulnerability Summary

#	Finding	Risk	Summary
1	SQL Injection	High	DVWA SQL Injection module accepted a classic tautology-based payload (1' OR '1'='1 #), indicating unsanitized input reaching the SQL query.
2	Reflected XSS	Medium	User-supplied script was reflected and executed in the browser, confirmed via a JavaScript alert popup.
3	Weak Default Credentials / Authentication Weakness	High	Default credentials (admin / password) were accepted, granting full authenticated access.
4	Parameter / Cookie Tampering	Medium	A client-controlled security cookie and a request parameter both influenced application behaviour when modified.
5	OS Command Injection	High	A command-injection payload was accepted by the Command Injection module without rejection, though the specific command output was not confirmed in the available evidence.
6	Directory / Endpoint Exposure	Low	Gobuster identified multiple reachable directories/endpoints not linked from the application's visible navigation.

Severity ratings above are qualitative (High / Medium / Low), based on the nature and potential impact of each finding within the DVWA lab context. No CVSS scores are claimed, as sufficient scoring information was not collected during this assessment.

13. Remediation & Security Recommendations

- Use parameterized SQL queries / prepared statements.
- Apply input validation on all user-supplied data.
- Apply proper output encoding before rendering user input.
- Implement a Content Security Policy.
- Use secure authentication practices.
- Remove or change default credentials.
- Enforce a strong password policy.
- Apply secure session management (session expiration, secure cookie flags).
- Perform all security-relevant authorization checks server-side.
- Treat client-controlled parameters and cookies as untrusted input.
- Use safe APIs for OS-level operations instead of direct shell execution.
- Apply directory and endpoint hardening; remove unnecessary files before deployment.
- Configure standard security headers (e.g. X-Content-Type-Options, X-Frame-Options).
- Implement logging and monitoring for suspicious activity.
- Perform regular VAPT assessments.
- Integrate security testing into the development lifecycle.

14. Conclusion

The Week 7 assessment provided practical exposure to web application VAPT using Burp Suite and complementary tools. A controlled DVWA environment was used to safely demonstrate reconnaissance, request interception, manual vulnerability testing, authentication testing, parameter manipulation, command injection testing, and endpoint discovery.

Multiple security weaknesses were identified and documented with evidence and remediation recommendations: SQL Injection, Reflected XSS, weak default credentials, parameter and cookie tampering, OS command injection, and directory/endpoint exposure. Burp Intruder configuration was also demonstrated, with the Community Edition's automated-attack limitation clearly noted rather than overstated.

All testing in this assessment was performed exclusively against the student's own authorized local DVWA lab; no public or third-party applications were tested. This exercise reinforced practical VAPT methodology — from proxy-based interception through to structured vulnerability documentation and remediation planning — using an industry-standard tool set.

15. Evidence / Screenshot References

Figure	Description	Section
1	DVWA login request captured in Burp Proxy HTTP history	Task 22
2	Burp Suite Site Map of the DVWA application	Task 23
3	SQL Injection request/response via Burp Repeater	Task 24 – SQL Injection
4	Reflected XSS alert popup confirming payload execution	Task 24 – Reflected XSS
5	Burp HTTP history — Reflected XSS request/response	Task 24 – Reflected XSS
6	DVWA login request/response during authentication testing	Task 24 – Authentication
7	Default credentials resent via Burp Repeater	Task 24 – Authentication
8	Security cookie tampering — modified response	Task 24 – Parameter/Cookie

Figure	Description	Section
		Tampering
9	Parameter tampering — modified ip value	Task 24 – Parameter/Cookie Tampering
10	Command Injection payload via Burp Repeater	Task 24 – Command Injection
11	Gobuster endpoint discovery output	Task 25